In [35]:
```
import pandas as pd
import numpy as np

df=pd.read_csv("C:/Users/HP/cluster.csv")
```

In [36]:
```
df
```

Out[36]:

	GDP per Capita	Life Expectancy	Literacy Rate	Unemployment Rate	Access to Clean Water	Poverty Rate	Country
0	1548.813504	55.684339	68.209932	15.046955	61.354741	38.558033	Country 1
1	1715.189366	50.187898	60.971013	21.778165	62.982823	30.117141	Country 2
2	1602.763376	56.176355	68.379449	17.700080	65.699649	33.599781	Country 3
3	1544.883183	56.120957	60.960984	22.351940	65.908728	37.299906	Country 4
4	1423.654799	56.169340	69.764595	24.621885	65.743252	31.716297	Country 5
...	...	...	...	...	...	...	...
95	3490.305347	78.817202	89.067333	9.610557	86.334615	14.979623	Country 96
96	3989.409777	72.724369	90.520783	5.447473	94.805801	8.621891	Country 97
97	3065.304207	73.790569	87.716528	7.029712	93.717857	9.706489	Country 98
98	3783.234438	73.742962	89.554441	5.121566	90.027208	8.782452	Country 99
99	3288.398497	77.487883	89.017135	6.713055	94.223480	14.795269	Country 100

100 rows × 7 columns

In [37]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 7 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   GDP per Capita         100 non-null    float64
 1   Life Expectancy        100 non-null    float64
 2   Literacy Rate          100 non-null    float64
 3   Unemployment Rate      100 non-null    float64
 4   Access to Clean Water  100 non-null    float64
 5   Poverty Rate           100 non-null    float64
 6   Country                100 non-null    object 
dtypes: float64(6), object(1)
memory usage: 5.6+ KB
```

In [38]:
```
list(df)
```

Out[38]:
```
['GDP per Capita',
 'Life Expectancy',
 'Literacy Rate',
 'Unemployment Rate',
 'Access to Clean Water',
 'Poverty Rate',
 'Country']
```

In [39]:
```
features = df.drop('Country', axis=1)
features
```

Out[39]:

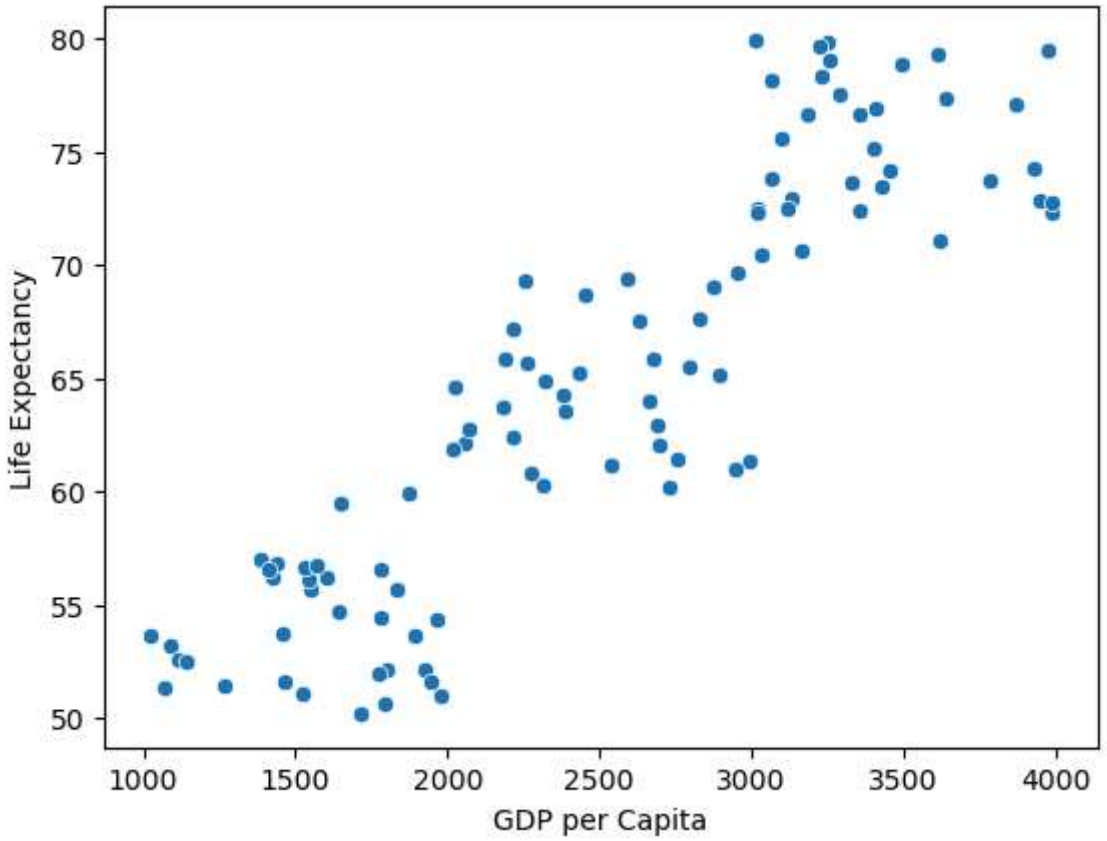
	GDP per Capita	Life Expectancy	Literacy Rate	Unemployment Rate	Access to Clean Water	Poverty Rate
0	1548.813504	55.684339	68.209932	15.046955	61.354741	38.558033
1	1715.189366	50.187898	60.971013	21.778165	62.982823	30.117141
2	1602.763376	56.176355	68.379449	17.700080	65.699649	33.599781
3	1544.883183	56.120957	60.960984	22.351940	65.908728	37.299906
4	1423.654799	56.169340	69.764595	24.621885	65.743252	31.716297
...	...	...	...	...	...	...
95	3490.305347	78.817202	89.067333	9.610557	86.334615	14.979623
96	3989.409777	72.724369	90.520783	5.447473	94.805801	8.621891
97	3065.304207	73.790569	87.716528	7.029712	93.717857	9.706489
98	3783.234438	73.742962	89.554441	5.121566	90.027208	8.782452
99	3288.398497	77.487883	89.017135	6.713055	94.223480	14.795269

100 rows × 6 columns

In [40]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

In [48]:
```
import seaborn as sns
sns.scatterplot(x=df['GDP per Capita'], y=df['Life Expectancy'])
```

Out[48]:
```
<AxesSubplot:xlabel='GDP per Capita', ylabel='Life Expectancy'>
```



In [73]:
```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)  # Example with 3 clusters
df['Cluster'] = kmeans.fit_predict(scaled_features)
```

In [ ]:

In [74]:
```
df
```

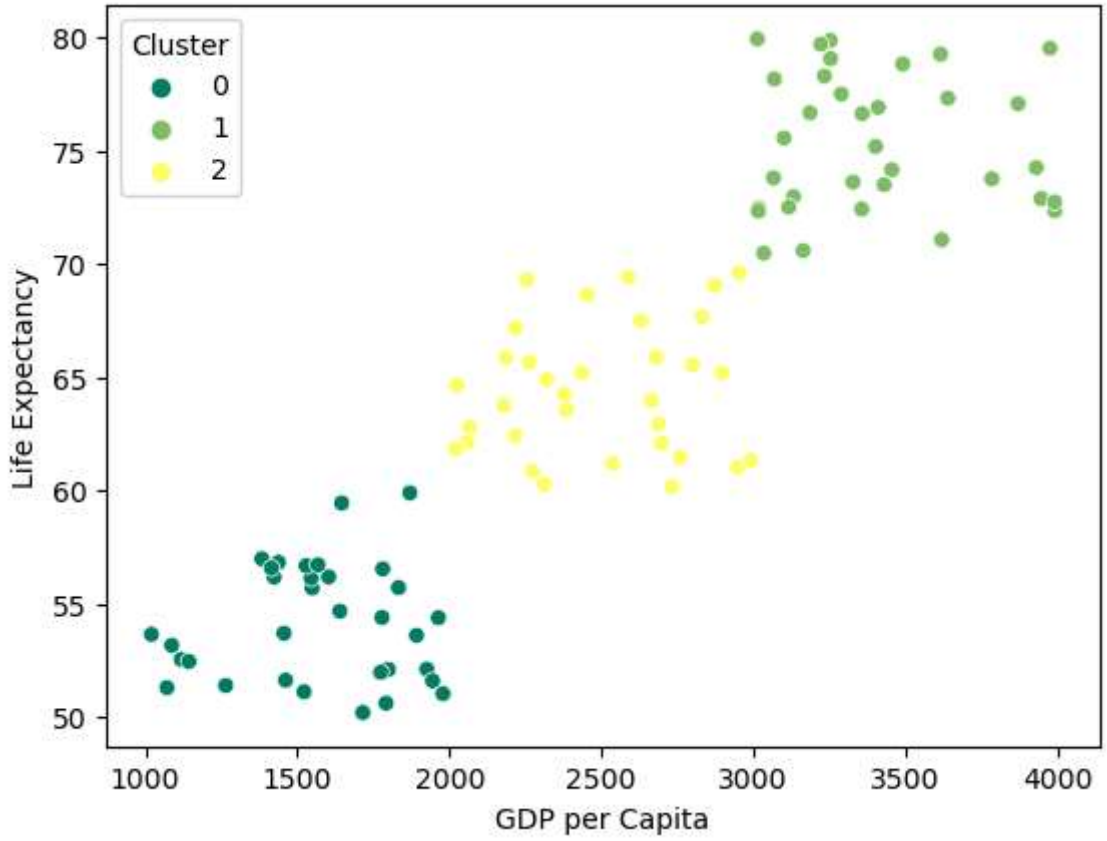
	GDP per capita	Life Expectancy	Literacy Rate	Unemployment Rate	Access to Clean Water	Poverty Rate	Country	Cluster
0	1548.813504	55.684339	68.209932	15.046955	61.354741	38.558033	Country 1	0
1	1715.189366	50.187898	60.971013	21.778165	62.982823	30.117141	Country 2	0
2	1602.763376	56.176355	68.379449	17.700080	65.699649	33.599781	Country 3	0
3	1544.883183	56.120957	60.960984	22.351940	65.908728	37.299906	Country 4	0
4	1423.654799	56.169340	69.764595	24.621885	65.743252	31.716297	Country 5	0
...	...	...	...	...	...	...	...	...
95	3490.305347	78.817202	89.067333	9.610557	86.334615	14.979623	Country 96	1
96	3989.409777	72.724369	90.520783	5.447473	94.805801	8.621891	Country 97	1
97	3065.304207	73.790569	87.716528	7.029712	93.717857	9.706489	Country 98	1
98	3783.234438	73.742962	89.554441	5.121566	90.027208	8.782452	Country 99	1
99	3288.398497	77.487883	89.017135	6.713055	94.223480	14.795269	Country 100	1

100 rows × 8 columns

In [75]:
```
1 sns.scatterplot(df['GDP per Capita'], df['Life Expectancy'], hue=df['Cluster'],palette="summer")
2
```

```
C:\Users\HP\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variabl
es as keyword args: x, y. From version 0.12, the only valid positional argument will be `data`, and passing
other arguments without an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

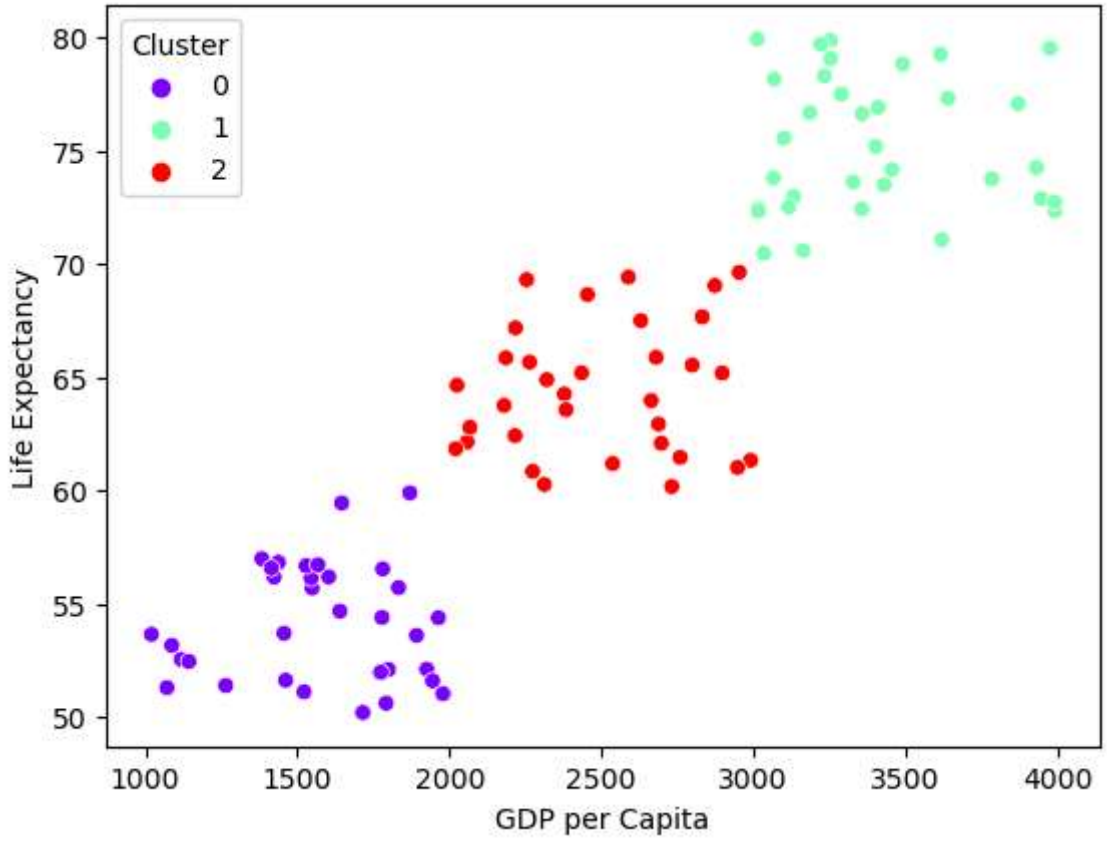
Out[75]: <AxesSubplot:xlabel='GDP per Capita', ylabel='Life Expectancy'>



In [77]:
```
sns.scatterplot(df['GDP per Capita'], df['Life Expectancy'], hue=df['Cluster'], palette='rainbow')
```

```
C:\Users\HP\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variabl
es as keyword args: x, y. From version 0.12, the only valid positional argument will be `data`, and passing
other arguments without an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

Out[77]:
```
<AxesSubplot:xlabel='GDP per Capita', ylabel='Life Expectancy'>
```



how can we know the number of clusters???

elbow method

In [30]:
```
scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)
```

In [82]:
```
# Apply the Elbow Method to determine the optimal number of clusters
inertia = []
k_values = range(1, 11)  # Check from 1 to 10 clusters

for k in k_values: #k=3
    kmeans = KMeans(n_clusters=k) #k=1
    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)
```

```
C:\Users\HP\anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1036: UserWarning: KMeans is known to hav
e a memory leak on Windows with MKL, when there are less chunks than available threads. You can avoid it by
setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

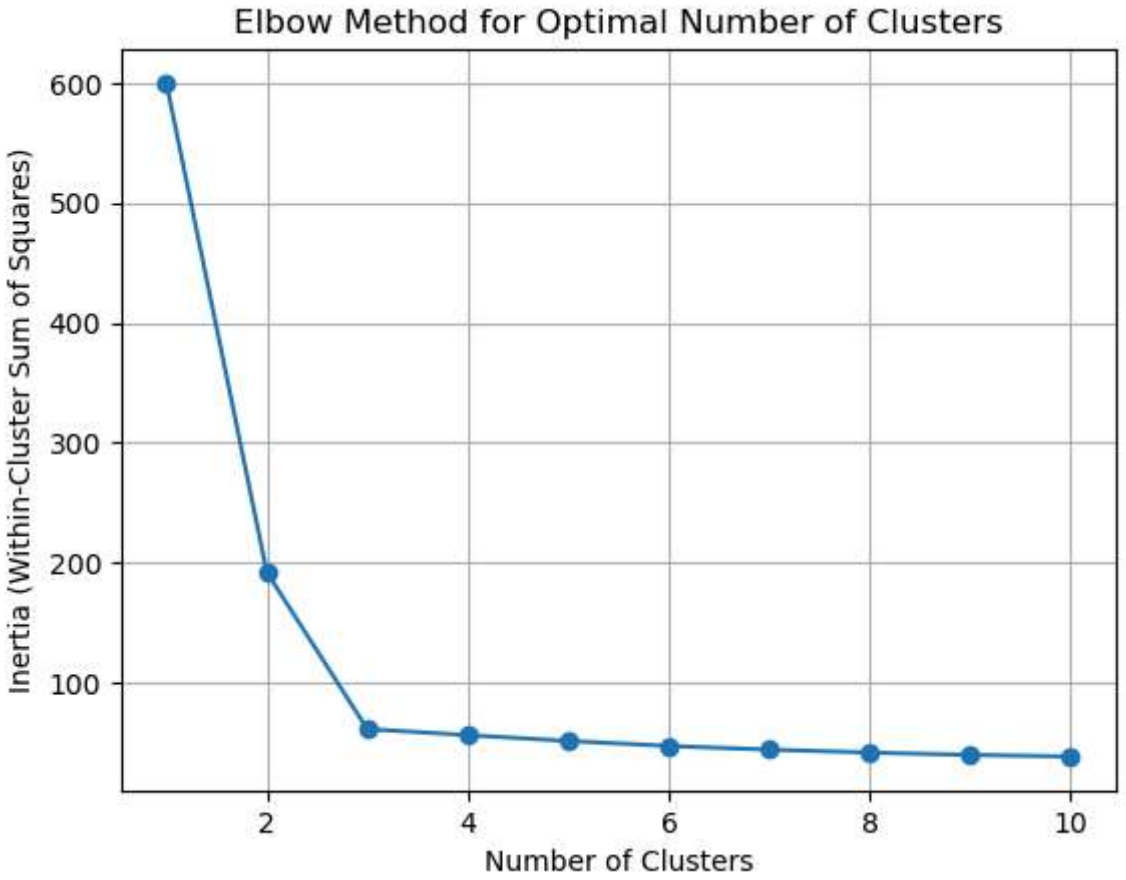
In [83]:
```
k_values
```

Out[83]:
```
range(1, 11)
```

In [84]:
```
inertia
```

Out[84]:
```
[600.0,
 191.78640430474806,
 60.95739240634884,
 55.8951541669937,
 51.051783817422816,
 46.848648748320954,
 43.787126523574514,
 41.3613520056738,
 39.38917294974313,
 38.04078997118016]
```

In [89]:
```
import matplotlib.pyplot as plt
# Plot the Elbow curve
plt.plot(k_values, inertia,marker='o')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia (Within-Cluster Sum of Squares)')
plt.title('Elbow Method for Optimal Number of Clusters')
plt.grid(True)
plt.show()
```



In [ ]:

In [79]:
```
x=[5,8]
```

In [80]:
```
x.append(10)
```

In [81]:
```
x
```

Out[81]:
```
[5, 8, 10]
```

In [ ]:
```

```